E-SHOP

Full-Stack E-Commerce Platform

Technical Documentation & Developer Guide

Version	Date
1.0.0	**March 2026**

Project Overview

A production-grade full-stack E-Commerce platform built with React 18, Node.js, MongoDB, and Tailwind CSS. Features include product catalog, shopping cart, checkout with Razorpay/COD payments, AI-powered churn prediction, and a complete dark-themed admin panel.

Stack: React 18 • Node.js / Express • MongoDB • Tailwind CSS • Zustand • Razorpay

Table of Contents

Table of Contents 2
1. Project Vision & Tech Stack 4
1.1 Core Problem Statement 4
1.2 Full Tech Stack 4
1.3 Architecture Overview 5
2. Initial Setup & Architecture 6
2.1 Repository Structure 6
2.2 Environment Variables 6
Frontend (.env) 6
Backend (.env) 6
2.3 Database Schema Design 7
User Model 7
Product Model 7
Order Model 7
3. Phase-wise Development Journey 9
Phase 1 — MVP (Basic UI + Authentication) 9
What was built: 9
Key files created: 9
Phase 2 — Core Features 9
Cart System 9
Checkout & Payments 9
Admin Panel 10
COD Payment Flow (Admin): 10
Phase 3 — Current State (UI/UX + Razorpay) 10
UI Redesign Completed: 10
Razorpay Integration Flow: 10
4. Critical Logic & Code Snippets 11
4.1 Authentication Middleware 11
4.2 Razorpay Integration (Frontend) 11
4.3 COD Payment Gate (Admin) 12
4.4 User confirm-payment Route (Backend) 12
4.5 Avatar Upload (Profile) 12
5. Bug Log & Solutions 14
6. Next Steps Roadmap 15
Immediate Priority — Fix Stock Deduction Bug 15
Priority 2 — Razorpay Signature Verification 15
Priority 3 — Production Deployment Checklist 15

Frontend (Vercel / Netlify) 16
Backend (Railway / Render / EC2) 16
Razorpay Live Mode 16
Future Features (Backlog) 16
Short-term 16
Medium-term 16
Long-term 16

1. Project Vision & Tech Stack

1.1 Core Problem Statement

E-Shop solves the challenge of building a complete, production-ready e-commerce system that covers every user and admin touchpoint — from product discovery and secure checkout to real-time order tracking and AI-powered customer retention analytics.

> Core Goal: A full-stack platform where a customer can browse products, pay securely, track orders — and an admin can manage everything from a sleek dark dashboard with AI churn insights.

1.2 Full Tech Stack

Technology	Version	Purpose / Why Chosen
React 18	`18.x`	Frontend UI — component-based, fast re-renders, large ecosystem
Vite	`5.x`	Build tool — significantly faster HMR than CRA, native ESM support
Tailwind CSS	`3.x`	Utility-first styling — rapid UI development, consistent design tokens
Zustand	`4.x`	State management — lightweight, no boilerplate vs Redux, perfect for cart/auth
React Query	`5.x`	Server state — caching, background refetch, loading/error states built-in
React Hook Form	`7.x`	Form management — minimal re-renders, built-in validation
Zod	`3.x`	Schema validation — type-safe validation shared frontend/backend
Node.js / Express	`20 / 4.x`	Backend API — non-blocking I/O, massive npm ecosystem
MongoDB	`7.x`	Database — flexible schema for product catalogs, document-based
Mongoose	`8.x`	ODM — schema enforcement, middleware hooks, query builder
JWT	`latest`	Authentication — stateless, scalable, no session storage needed
Cloudinary	`latest`	Image storage — CDN delivery, automatic optimization, free tier
Razorpay	`v1`	Payments — India-first, supports UPI/cards/netbanking, test mode
Python / Flask	`3.11`	ML microservice — churn prediction model, separate service
Multer	`1.x`	File uploads — multipart/form-data handling for product/avatar images
bcryptjs	`2.x`	Password hashing — salt rounds for secure credential storage

1.3 Architecture Overview

```
┌──────────────────────────────────────────────────────────┐
│                    CLIENT (React/Vite)                   │
│  Pages → Components → Zustand Store → React Query        │
│  Tailwind CSS  •  React Hook Form  •  Zod Validation     │
└──────────────────────────────────────────────────────────┘
                       │ REST API (axios)
                       ▼
┌──────────────────────────────────────────────────────────┐
│             BACKEND (Node.js / Express)                  │
│  Routes → Middleware → Controllers → Services            │
│  JWT Auth  •  Multer  •  Cloudinary  •  Razorpay         │
└──────────────────────────────────────────────────────────┘
                       │ Mongoose ODM
                       ▼
┌──────────────────────────────────────────────────────────┐
│                   MongoDB Atlas                          │
│  Users • Products • Orders • Cart • AdminLogs            │
└──────────────────────────────────────────────────────────┘
                       │ HTTP
                       ▼
┌──────────────────────────────────────────────────────────┐
│          ML Microservice (Python / Flask)                │
│         Churn Prediction  •  scikit-learn                │
└──────────────────────────────────────────────────────────┘
```

2. Initial Setup & Architecture

2.1 Repository Structure

```
my project/
├── frontend/                    # React + Vite
│   ├── src/
│   │   ├── components/
│   │   │   ├── admin/           # AdminSidebar, StatsCard, ProductForm
│   │   │   ├── cart/            # CartItem, CartSummary
│   │   │   ├── checkout/        # ShippingAddress, PaymentMethod...
│   │   │   ├── common/          # Header, Footer, Loader, HeroSlider
│   │   │   └── product/         # ProductCard, ProductGrid, ProductFilter
│   │   ├── pages/
│   │   │   ├── admin/           # Dashboard, ProductsManagement...
│   │   │   ├── Cart.jsx
│   │   │   ├── Checkout.jsx
│   │   │   ├── Home.jsx
│   │   │   ├── Login.jsx
│   │   │   ├── Orders.jsx
│   │   │   └── Profile.jsx
│   │   ├── services/            # api.js, authService, orderService...
│   │   ├── store/               # authStore.js, cartStore.js (Zustand)
│   │   ├── utils/               # constants.js, formatters.js
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── router.jsx
│   ├── .env                     # VITE_API_URL, VITE_RAZORPAY_KEY_ID
│   └── package.json
│
└── backend/                     # Node.js + Express
    └── src/
        ├── config/              # database.js, cloudinary.js, constants.js
        ├── controllers/         # authController, orderController...
        ├── middlewares/         # auth.js, admin.js, validation.js
        ├── models/              # User.js, Product.js, Order.js, Cart.js
        ├── routes/              # authRoutes, orderRoutes...
        ├── services/            # paymentService.js, emailService.js
        ├── utils/               # helpers.js, jwt.js, logger.js
        ├── app.js
        ├── server.js
        └── .env
```

2.2 Environment Variables

Frontend (.env)

```
# frontend/.env
VITE_API_URL=http://localhost:5000/api
VITE_RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxxxxxxxxxx
```

Backend (.env)

```
# backend/.env
NODE_ENV=development
PORT=5000
```

```
# MongoDB
MONGODB_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/eshop

# JWT
JWT_SECRET=your_super_secret_key_here
JWT_EXPIRE=7d
JWT_REFRESH_EXPIRE=30d

# Cloudinary
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# Razorpay (Test)
RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxxxxxxxxx
RAZORPAY_KEY_SECRET=your_razorpay_secret
```

2.3 Database Schema Design

User Model

```
User {
  name:      String  (required, 2-50 chars)
  email:     String  (unique, validated)
  password:  String  (hashed, bcrypt, select:false)
  role:      enum    ["user", "admin"]
  avatar:    { url: String, publicId: String }   // Cloudinary
  address:   { street, city, state, zipCode, country }
  phone:     String  (10 digits)
  isActive:  Boolean (soft delete)
  lastLogin: Date
}

// □ IMPORTANT: avatar must be Object, NOT String
// Bug we hit: avatar: { type: String } caused [object Object] saves
```

Product Model

```
Product {
  name:           String  (required)
  description:    String  (min 10 chars)
  price:          Number  (required)
  compareAtPrice: Number  (for discount display)
  category:       enum    [electronics|clothing|home|books|beauty|sports|other]
  brand:          String
  images:         [{ url: String, publicId: String }]
  stock:          Number
  sku:            String  (auto-generated)
  isFeatured:     Boolean (shown on Home page)
  isActive:       Boolean
  ratings:        { average: Number, count: Number }
  specifications: [{ key: String, value: String }]
}
```

Order Model

```
Order {
```

```
  user:            ObjectId (ref: User)
  orderNumber:     String  (auto-generated, unique)
  items:           [{ product, name, price, quantity, image, sku }]
  shippingAddress: { street, city, state, zipCode, country, phone }
  payment: {
    method:  enum  [cash_on_delivery|razorpay|stripe|paypal|credit_card|debit_card]
    //              ^^^ razorpay must be in enum — Bug #3
    status:  enum  [pending|completed|failed]
    transactionId: String
    paidAt:  Date
  }
  status:          enum  [pending|processing|shipped|delivered|cancelled]
  subtotal, tax, shippingCost, discount, total: Number
  trackingNumber, courier: String
  notes:           String
  estimatedDelivery: Date
}
```

3. Phase-wise Development Journey

Phase 1 — MVP (Basic UI + Authentication)

The first phase established the core foundation: project scaffolding, user authentication, and basic product display.

What was built:

- Vite + React project setup with Tailwind CSS
- Express backend with MongoDB connection
- JWT authentication (register, login, logout, refresh tokens)
- Basic product CRUD (Create, Read, Update, Delete)
- Simple product listing page and product detail
- Protected routes with ProtectedRoute and AdminRoute components

Key files created:

- authController.js — register/login/getMe endpoints
- authStore.js — Zustand store with persist middleware
- ProtectedRoute.jsx & AdminRoute.jsx — route guards
- Header.jsx — navigation with auth state

Phase 2 — Core Features

Phase 2 implemented all the main e-commerce functionality: cart, checkout, order management, and the full admin panel.

Cart System

- Cart model with user association in MongoDB
- cartStore.js — Zustand with cart operations (add, remove, update, clear)
- CartItem component with quantity controls and optimistic updates
- CartSummary with free shipping progress bar

Checkout & Payments

- Multi-step checkout form (Shipping → Billing → Payment → Notes)
- React Hook Form + Zod validation
- Cash on Delivery — always available
- Razorpay integration — popup with UPI/cards/netbanking
- Order creation from cart with stock validation

Admin Panel

- Dark theme dashboard with StatsCard components
- Products management — CRUD with image upload via Cloudinary
- Orders management — status advancement with payment gate
- Users management — role toggle, activate/deactivate
- Churn Analytics — AI predictions with risk visualization

COD Payment Flow (Admin):

- Order placed → payment.status = "pending"
- Admin sees BanknotesIcon on COD orders
- Admin clicks "Mark as Paid" → paymentStatus: "completed" sent to PUT /:id/payment
- Truck button unlocks → admin can advance order status

Phase 3 — Current State (UI/UX + Razorpay)

Phase 3 focused on production-quality UI redesign and live payment integration.

UI Redesign Completed:

- ProductCard — hover zoom, quick-add slide, rating stars, badges
- HeroSlider — crossfade, per-slide accent color, progress bar
- Home.jsx — categories, featured products, testimonials, promo banner
- Header — scroll-aware, active link highlighting, avatar dropdown
- Footer — dark theme, newsletter input, social icons
- Login/Register — split-screen layout, password strength bar
- Profile — 5-tab layout with avatar upload, address, orders, danger zone
- ProductDetail — image zoom, Buy Now button, tabs, related products
- Cart/Checkout — free shipping bar, payment cards, 4-step progress
- Orders/OrderDetail — visual tracker, cancel modal, filter tabs
- Admin Panel — full dark Vercel/Linear aesthetic

Razorpay Integration Flow:

1. User selects Razorpay on checkout
2. handleRazorpay() loads Razorpay script dynamically
3. createFromCart() creates order in MongoDB
4. Razorpay popup opens with order amount
5. User pays via UPI/card/netbanking
6. handler() fires — calls POST /orders/:id/confirm-payment
7. payment.status = "completed", transactionId saved
8. Cart cleared, navigate to order page

4. Critical Logic & Code Snippets

4.1 Authentication Middleware

The auth middleware extracts userId from JWT and attaches it to req. Critical: uses req.userId (not req.user.id) throughout.

```
// middlewares/auth.js
exports.authenticate = async (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return next(new AppError("No token provided", 401));

  const decoded = jwt.verify(token, process.env.JWT SECRET);
  req.userId = decoded.id;   // ← Always req.userId, never req.user.id
  req.user = await User.findById(decoded.id);
  next();
};
```

> Bug avoided: Multer filename function used req.user.id which crashed because middleware sets req.userId. Always use req.userId.

4.2 Razorpay Integration (Frontend)

```
// pages/Checkout.jsx — Razorpay flow
const loadRazorpayScript = () => new Promise((resolve) => {
  if (window.Razorpay) { resolve(true); return; }
  const script = document.createElement("script");
  script.src = "https://checkout.razorpay.com/v1/checkout.js";
  script.onload = () => resolve(true);
  document.body.appendChild(script);
});

const handleRazorpay = async (data) => {
  await loadRazorpayScript();
  const orderResponse = await orderService.createFromCart(orderData);
  const order = orderResponse.data.order;

  const options = {
    key: import.meta.env.VITE_RAZORPAY_KEY_ID,
    amount: Math.round(order.total * 100),  // paise
    currency: "INR",
    name: "E-Shop",
    prefill: { name: user?.name, email: user?.email },
    theme: { color: "#4f46e5" },
    handler: async (response) => {
      // ✅ Use user-accessible route, NOT admin route
      await orderService.confirmPayment(order. id, {
        razorpay_payment_id: response.razorpay_payment_id,
      });
      await clearCart();
      navigate(`/orders/${order._id}`);
    },
  };
  new window.Razorpay(options).open();
};
```

4.3 COD Payment Gate (Admin)

The truck button (advance order status) is gated behind payment completion. Uses a flexible isCODOrder helper to match any DB format.

```
// Helper — matches any COD variant stored in DB
const isCODOrder = (order) => {
  const method = order.payment?.method?.toLowerCase() || "";
  return method.includes("cash") || method.includes("cod");
};

// In render:
{order.payment?.status === "completed"
  ? <TruckIcon onClick={() => handleStatusUpdate(order)} />  // Active
  : <DisabledTruckWithTooltip message="Mark paid first" />   // Locked
}

// COD Mark as Paid button
{isCODOrder(order) && order.payment?.status !== "completed" && (
  <BanknotesIcon onClick={() => markPaidMutation.mutate(order._id)} />
)}
```

4.4 User confirm-payment Route (Backend)

A user-accessible route that allows payment confirmation without admin privileges — critical for Razorpay flow.

```
// routes/orderRoutes.js
router.post("/:id/confirm-payment", authenticate,
  catchAsync(async (req, res, next) => {
    const { razorpay_payment_id } = req.body;

    // Find order belonging to THIS user only
    const order = await Order.findOne({
      _id: req.params.id,
      user: req.userId          // ← Security: user owns order
    });
    if (!order) return next(new Error("Order not found"));

    order.payment.status = "completed";
    order.payment.transactionId = razorpay_payment_id;
    order.payment.paidAt = new Date();
    await order.save();

    res.status(200).json({ success: true, order });
  })
);
```

4.5 Avatar Upload (Profile)

Avatar saved as an object {url, publicId} in MongoDB — NOT a plain string.

```
// ✅ CORRECT — User.js model
avatar: {
  url:       { type: String, default: null },
  publicId:  { type: String, default: null }
}

// ❌ WRONG — caused [object Object] saves
// avatar: { type: String, default: null }

// Frontend read (handles both old String and new Object):
const avatarSrc =
  typeof user?.avatar === "string" ? user.avatar : user?.avatar?.url;
```

5. Bug Log & Solutions

A complete log of all major bugs encountered during development and their resolutions.

#	Error	Cause	Fix
1	`Multer crash: req.user.id undefined`	Auth middleware sets req.userId not req.user.id. Multer filename function used req.user.id.	Change req.user.id → req.userId in multer filename config in userRoutes.js
2	`Avatar saves as [object Object]`	User.js model had avatar: { type: String } but controller saved { url, publicId } object.	Fix model: avatar: { url: String, publicId: String }. Frontend: read user?.avatar?.url
3	`Order 500: razorpay not valid enum`	Order.js payment.method enum did not include "razorpay" value.	Add "razorpay" to enum array in Order.js model. Restart backend.
4	`Payment 403 Forbidden`	Frontend called PUT /orders/:id/payment which is admin-only route after Razorpay success.	Created POST /orders/:id/confirm-payment route accessible by authenticated user only.
5	`Insufficient stock error loop`	Each failed order attempt cut product stock. Stock hit 0 causing all future orders to fail.	Fix: move stock decrement AFTER Order.create(). Short term: restore stock in Admin Products.
6	`COD truck button always locked`	isCODOrder() checked exact match "cash_on_delivery" but DB stored different format.	Use flexible helper: method.includes("cash") || method.includes("cod") to match any variant.
7	`markPaidMutation sent wrong key`	Frontend sent { status: "completed" } but updatePaymentStatus controller expected { paymentStatus }.	Fix payload: { paymentStatus: "completed" } in markPaidMutation.mutationFn.
8	`HMR: router export incompatible`	AdminLayout component defined inline in router.jsx caused Vite Fast Refresh failure.	Move AdminLayout to its own file src/components/common/AdminLayout.jsx and import it.
9	`Admin panel shows public Header/Footer`	Admin routes nested inside App.jsx route which wraps all pages with Header/Footer.	Move /admin route to top-level in router.jsx, outside App route. Admin gets own layout.
10	`Profile photo not persisting`	updateProfile() in authStore calls authService.updateDetails internally, not the FormData endpoint. Zustand user never updated.	Call loadUser() after successful profile save to re-fetch fresh user from backend into store.

6. Next Steps Roadmap

Immediate Priority — Fix Stock Deduction Bug

> □ CRITICAL: Currently stock is cut on every order attempt, even failed ones. This causes "Insufficient stock" errors on retries.

Fix in orderController.js — createOrderFromCart:

```
// STEP 1: Remove stock decrement from the loop
for (const cartItem of cart.items) {
  // ... validate, build orderItems ...
  // ❌DELETE THESE TWO LINES:
  // product.stock -= cartItem.quantity;
  // await product.save();
}

// STEP 2: Add stock decrement AFTER Order.create()
const order = await Order.create({ ... });

// ✅Only cut stock after DB order is confirmed
for (const item of order.items) {
  await Product.findByIdAndUpdate(item.product, {
    $inc: { stock: -item.quantity }
  });
}
```

Priority 2 — Razorpay Signature Verification

Currently Razorpay payments are confirmed without server-side signature verification. For production, implement proper HMAC verification.

```
// In confirm-payment route, add verification:
const crypto = require("crypto");

const body = razorpay_order_id + "|" + razorpay_payment_id;
const expectedSig = crypto
  .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET)
  .update(body)
  .digest("hex");

if (expectedSig !== razorpay_signature) {
  return next(new AppError("Invalid payment signature", 400));
}

// This requires sending razorpay_order_id from frontend too
// Generate Razorpay order server-side first (POST /orders/:id/razorpay/create)
```

Priority 3 — Production Deployment Checklist

Frontend (Vercel / Netlify)

- Change VITE_API_URL to production backend URL
- Change VITE_RAZORPAY_KEY_ID to rzp_live_xxx (after KYC)
- Add _redirects file for SPA routing: /* /index.html 200

Backend (Railway / Render / EC2)

- Set NODE_ENV=production
- Set all .env variables in hosting dashboard
- Enable CORS only for your frontend domain
- Add rate limiting (express-rate-limit) on auth routes
- Set up MongoDB Atlas IP whitelist for production server

Razorpay Live Mode

- Complete KYC on Razorpay dashboard
- Replace test keys with live keys
- Enable HTTPS (required for live payments)
- Add webhook endpoint for payment status updates

Future Features (Backlog)

Short-term

- Product search with debounce
- Order tracking email notifications (emailService.js already exists)
- Coupon/discount code system
- Product reviews — real API (currently uses mock data)

Medium-term

- Wishlist functionality (backend + frontend)
- Push notifications for order status changes
- Analytics charts on admin dashboard (recharts)
- Multi-image reordering on product form

Long-term

- PWA support — offline browsing, push notifications
- Vendor/seller portal for multi-vendor marketplace
- Real-time chat support (Socket.io)
- ML recommendation engine (collaborative filtering)

E-Shop Technical Documentation

Built with React 18 • Node.js • MongoDB • Razorpay

Version 1.0 — March 2026